

SOFTWARE CONSTRUCTION AND DEVELOPMENT

Engr. Muhammad Umer Haroon

SOFTWARE CONSTRUCTION AND DEVELOPMENT

WEEK 8

RECALL LAST WEEKS

Every programmer eventually learns a painful lesson. The code that worked perfectly on your laptop crashes the moment a real user touches it. The file you expected to exist is missing. The number you expected is a word. The network you expected to respond goes silent.

Defensive programming is the discipline of writing code that anticipates these problems before they happen.

Exception handling is the mechanism Python provides to deal with problems when they do happen.

Together, they form one of the most important skills in software construction, because they determine whether your software fails gracefully or fails catastrophically.

What Is Defensive Programming

The opposite of defensive programming is optimistic programming, where the code assumes everything will be fine. Optimistic code is shorter and easier to write, but it fails badly when reality does not match expectations.

Why Defensive Programming Matters in Software Construction

CONCERN	HOW DEFENSIVE PROGRAMMING HELPS
User input	Validates before using, prevents crashes
File operations	Handles missing or unreadable files
Network calls	Handles timeouts and connection failures
Database queries	Handles missing rows and constraint violations
Third-party APIs	Handles unexpected responses and rate limits
Configuration	Handles missing or malformed settings
Data processing	Handles malformed records without stopping the pipeline
Production systems	Prevents small errors from becoming outages

Any problem here ?

```
def calculate_age(year_born):
    current_year = 2024
    return current_year - year_born

age = calculate_age("1995")
print(age)  # TypeError
```

Defensive Version

```
def calculate_age(year_born):
    if not isinstance(year_born, int):
        raise ValueError("Year must be an integer")
    if year_born < 1900 or year_born > 2024:
        raise ValueError("Year must be between 1900 and 2024")
    return 2024 - year_born
```

The Three Enemies of Reliable Software

Defensive programming is the discipline of anticipating all three enemies and designing your code to survive them

ENEMY	EXAMPLE	CONSEQUENCE
Bad input	User types "abc" where a number is expected	Crash or wrong result
Bad environment	File missing, network down, disk full	Crash or data loss
Bad assumptions	Developer assumes a list is never empty	Crash or wrong result

Predict the output of each line.

```
def calculate_discount(price, discount_percent):
    return price - (price * discount_percent / 100)

print(calculate_discount(100, 10))     # what happens ?
print(calculate_discount(100, "10"))   # what happens?
print(calculate_discount("100", 10))   # what happens?
print(calculate_discount(100, 150))    # what happens?
```

- The first works.
- The second raises a TypeError.
- The third raises a TypeError.
- The fourth returns a negative price, which is wrong but does not crash.

This exercise shows that some problems cause crashes and some cause silent wrong results, and both are dangerous.

- Defensive programming means writing code that assumes things will go wrong and handles them gracefully instead of crashing or producing wrong results. It is not about being paranoid. It is about being professional.
- Think of it as the difference between a driver who assumes everyone will follow the rules and a driver who watches for mistakes. The defensive driver still drives normally, but they are prepared for the unexpected.

The Three Layers of Defense

LAYER	PURPOSE
Input validation	Reject bad data at the boundary
Internal checks/ Assertion	Verify assumptions inside functions
Exception handling	Recover from failures that cannot be prevented

The first two layers prevent errors. The third layer handles errors that cannot be prevented. A robust system uses all three.

- The key distinction is this. Input validation is about data coming from outside your code.
- Exception handling is about operations that can fail at runtime, such as file access or network calls.
- Assertions are about internal assumptions that should always be true if your code is correct.
- These three layers are not the same thing, and they are not interchangeable. A common mistake is to use exceptions for input validation or assertions for runtime errors. Each layer has its proper place.

Analogy

Imagine an airport.

Input validation is the security check at the entrance, where bad items are stopped before they enter.

Exception handling is the emergency exit plan, for when something unexpected happens inside.

Assertions are the maintenance checks, ensuring the equipment is working correctly before passengers arrive. Each one serves a different purpose and none can replace the others.

Input Validation, the First Line of Defense

Input validation is checking data as it enters your system. It is the cheapest and most effective defense because bad data caught at the boundary never reaches the rest of your code.

Validation happens at the boundary, which means at the point where data enters from the outside world, such as a function that receives user input, a function that reads from a file, or an API endpoint that receives a request.

```
def calculate_discount(price, discount_percent):
    return price - (price * discount_percent / 100)
```

This function assumes price is a number and discount_percent is a number between 0 and 100. If either assumption is wrong, the function crashes or produces a wrong result.

Validated Version

```
def calculate_discount(price, discount_percent):
    if not isinstance(price, (int, float)):
        raise ValueError("Price must be a number")
    if price < 0:
        raise ValueError("Price cannot be negative")
    if not isinstance(discount_percent, (int, float)):
        raise ValueError("Discount must be a number")
    if discount_percent < 0 or discount_percent > 100:
        raise ValueError("Discount must be between 0 and 100")
    return price - (price * discount_percent / 100)
```

Now the function checks its assumptions before doing any work. If the input is wrong, it raises a clear error instead of producing a TypeError or a silent wrong result.

The Validation Pattern

A consistent validation pattern has three steps. First, check the type. Second, check the value range or format. Third, return a clear result or raise a clear error.

```
def validate_username(username):
    if not isinstance(username, str):
        return False, "Username must be a string"
    if len(username) < 3:
        return False, "Username must be at least 3 characters"
    if len(username) > 20:
        return False, "Username must be at most 20 characters"
    if not username.isalnum():
        return False, "Username must contain only letters and numbers"
    return True, "Valid"

result, message = validate_username("ali123")
print(result, message)   # True Valid

result, message = validate_username("al")
print(result, message)   # False Username must be at least 3 characters
```

This pattern returns both a boolean and a message, which is useful when the caller needs to show the error to a user.

Validation at the Boundary

The most important rule of validation is to validate at the boundary, not deep inside your code. Data should be validated once, as it enters, and then trusted throughout the rest of the system.

```
def create_user(username, age):
    # Validate at the boundary
    if not isinstance(username, str) or len(username) < 3:
        raise ValueError("Invalid username")
    if not isinstance(age, int) or age < 0 or age > 120:
        raise ValueError("Invalid age")

    # From here on, username and age are trusted
    user = {"username": username, "age": age}
    save_to_database(user)
    send_welcome_email(user)
    return user
```

The functions save_to_database and send_welcome_email do not need to validate again because the data has already been validated. This keeps validation in one place and avoids redundant checks.

Common Validation Mistakes

MISTAKE	PROBLEM	FIX
Validating after using the data	Crash happens before check	Validate first
Validating in every function	Redundant checks, scattered logic	Validate at the boundary only
Using exceptions for control flow	Slow and confusing	Return booleans for expected cases
Not validating at all	Crashes on bad input	Always validate external input
Trusting user input	Security vulnerabilities	Treat all external input as untrusted

Input validation is like the security guard at a ground. The guard checks IDs at the door, not inside the ground. Once you are in, you are trusted. If the guard checked every person every time they moved, the ground would be chaos. The same is true for validation. Check once, at the boundary, and trust the data afterward.

What Exception Handling Is

- Exception handling is Python's mechanism for dealing with problems that happen at runtime. Unlike input validation, which prevents bad data from entering, exception handling deals with failures that cannot be prevented, such as a file that does not exist, a network that goes down, or a database that is unavailable.
- The key distinction from validation is this. Validation is about data coming in. Exception handling is about operations that can fail for reasons outside your control.

Basic try except Block

```
try:
    with open("data.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("The file does not exist")
except PermissionError:
    print("You do not have permission to read this file")
```

The try block contains code that might fail. The except blocks contain code that handles specific failures. If the file exists and is readable, the except blocks are skipped. If the file is missing, the FileNotFoundError block runs.

BLOCK	WHEN IT RUNS
try	Always, contains the code that might fail
except	Only if a matching exception is raised
else	Only if no exception occurred
finally	Always, whether or not an exception occurred

```
try:
    result = risky_operation()
except SpecificError as e:
    handle_error(e)
except AnotherError as e:
    handle_another_error(e)
else:
    print("No error occurred")
finally:
    cleanup()
```

The finally block is for cleanup. It runs even if the try block returns early or raises an exception. It is used for releasing resources such as files, locks, and connections.

Catching Specific Exceptions

```
# Bad: catches everything, hides bugs
try:
    result = process_data(data)
except Exception:
    pass

# Good: catches only what you expect
try:
    result = process_data(data)
except ValueError as e:
    print(f"Invalid data: {e}")
```

The first version silently swallows every error, including bugs in your own code. The second version handles the expected error but lets unexpected errors surface so you can fix them.

The Exception Hierarchy

Python exceptions inherit from each other in a hierarchy. Catching a parent class catches all its children.

```
BaseException
    Exception
        ArithmeticError
            ZeroDivisionError
        LookupError
            IndexError
            KeyError
        ValueError
        TypeError
        OSError
            FileNotFoundError
            PermissionError
```

This means **except Exception** catches almost everything, while **except FileNotFoundError** catches only one specific case. The more specific the exception, the safer the code.

When to Catch and When to Let Go

- The rule is simple. Catch an exception only when you can do something useful with it. If you cannot fix the problem or inform the user(not necessarily end user), let the exception propagate.

SITUATION	ACTION
You can recover from the error	Catch and recover
You can show a friendly message to the user	Catch and display
You can log the error and continue	Catch and log
You cannot do anything about it	Let it propagate
It is a bug in your code	Let it propagate

How does error propagation work?

Imagine a user tries to divide two numbers.

1. A function encounters an error
Example: division by zero
2. The error propagates
The current function doesn't handle it
3. A higher-level handler catches it
It logs the error and decides what to do
4. The user sees a friendly message
"Sorry, something went wrong. Please try again."

Notice that the error propagated through the program, but the user never saw the technical error itself.

Sometimes you need to raise an exception yourself to signal that something is wrong.

```
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive")
    if amount > balance:
        raise ValueError("Insufficient funds")
    return balance - amount
```

How Validation and Exception Handling Work Together

Input validation and exception handling are complementary, not interchangeable.

- Validation prevents bad data from entering.
- Exception handling deals with operations that can fail.

A robust system uses both.

```
def read_student_marks(filename):
    # Input validation: check the filename
    if not isinstance(filename, str) or not filename:
        raise ValueError("Filename must be a non empty string")

    # Exception handling: deal with file operations that can fail
    try:
        with open(filename, "r") as file:
            lines = file.readlines()
    except FileNotFoundError:
        raise FileNotFoundError(f"File not found: {filename}")
    except PermissionError:
        raise PermissionError(f"Permission denied: {filename}")

    # Process the data
    marks = []
    for line in lines:
        try:
            marks.append(int(line.strip()))
        except ValueError:
            continue
    return marks
```

READ IT YOUR SELF

Silence the complaint.

Execute the effort.

For queries

Office : B05, RSB

Counselling Hours :

Monday	11:00 am -12:00 pm
Tuesday	12:00 pm -1:00 pm
Thursday	10:15 am -11:15 am

mumer.haroon@imsciences.edu.pk

Muhammad Umer Haroon | LinkedIn